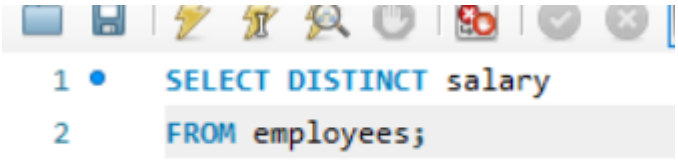
MySQL Assignment 2 -Querying Data

Clause & Operators:

1. **DISTINCT VALUES:**



```
1  SELECT DISTINCT salary
2  FROM employees;
```

2. **ALIAS (AS):**

Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.

```
SELECT
    age AS Employee_Age,
    salary AS Employee_Salary
FROM employees;
```

3. **WHERE CLAUSE & OPERATORS:**

Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01

```
SELECT *
FROM employees
WHERE salary > 50000
  AND hire_date < '2016-01-01';
```

Find the employee whose designation is missing and fill it with "Data Scientist".

```
  SELECT *
FROM employees
WHERE designation IS NULL;
UPDATE employees
SET designation = 'Data Scientist'
WHERE designation IS NULL;
```

Sorting and Grouping Data:

1. ORDER BY:

Find employees sorted by department ID in ascending order and salary in descending order.

```
SELECT *
FROM employees
ORDER BY department_id ASC, salary DESC;
```

2. LIMIT:

Display the first 5 employees hired in the year 2018

```
SELECT *
FROM employees
WHERE YEAR(hire_date) = 2018
ORDER BY hire_date
LIMIT 5;
```

3. AGGREGATE FUNCTIONS:

Calculate the sum of all salaries in the Finance department.

```sql
SELECT SUM(e.salary) AS Total_Finance_Salary
FROM employees e
JOIN departments d
    ON e.department_id = d.department_id
WHERE d.department_name = 'Finance';
```

Find the minimum age among all employees.

```sql
SELECT MIN(age) AS Minimum_Age
FROM employees;
```

4. **GROUP BY:**

List the maximum salary for each location

```sql
SELECT
    l.location,
    MAX(e.salary) AS maximum_salary
FROM employees e
JOIN location l
    ON e.location_id = l.location_id
GROUP BY l.location;
```

Calculate the average salary for each designation containing the word 'Analyst'.

```sql
SELECT
    designation,
    AVG(salary) AS average_salary
FROM employees
WHERE designation LIKE '%Analyst%'
GROUP BY designation;
```

5. HAVING:

Find departments with less than 3 employees

```sql
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count
FROM departments d
JOIN employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.employee_id) < 3;
```

Find locations with female employees whose average age is below 30

```sql
SELECT
    l.location,
    AVG(e.age) AS average_age
FROM employees e
JOIN location l
    ON e.location_id = l.location_id
WHERE e.gender = 'F'
GROUP BY l.location
HAVING AVG(e.age) < 30;
```

Joins:

1. **INNER JOIN:**

List employee names, their designations, and department names where employees are assigned to a department.

```sql
SELECT
    e.employee_name,
    e.designation,
    d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

2. **LEFT JOIN:**

- List all departments along with the total number of employees in each

Department, including departments with no employees.

```sql
    SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS total_employees
FROM departments d
LEFT JOIN employees e
    ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY d.department_id;
```

3. **RIGHT JOIN:**

Display all locations along with the names of employees assigned to each location. If no employees are assigned to a location, display NULL for employee name.

```sql
SELECT
    l.location,
    e.employee_name
FROM employees e
RIGHT JOIN location l
    ON e.location_id = l.location_id
ORDER BY l.location, e.employee_name;
```

4. **CROSS JOIN**

Show all possible combinations of departments and locations.

```sql
SELECT
    d.department_name,
    l.location
FROM departments d
CROSS JOIN location l
ORDER BY d.department_name, l.location;
```

5**. SELF JOIN:**

Show pairs of employees working in the same department, excluding self-pairs.

```sql
SELECT
    e1.employee_name AS Employee_1,
    e2.employee_name AS Employee_2,
    e1.department_id
FROM employees e1
JOIN employees e2
    ON e1.department_id = e2.department_id
   AND e1.employee_id < e2.employee_id;
```

Windows function

Write a window function query to rank employees by salary using rank(). ● Write a window function query to rank employees by salary within each department using DENSE_RANK()

```
SELECT
 employee_id,
 employee_name,
 salary,
 RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employees;
```

- Write a window function query, Running total salary by department

```
SELECT
    employee_id,
    employee_name,
    department_id,
    salary,
    SUM(salary) OVER (
        PARTITION BY department_id
        ORDER BY salary
    ) AS running_total_salary
FROM employees;
```